UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"
Campus de São José dos Campos
Instituto de Ciência e Tecnologia

Regression Models

Prof. Dr. Rogério Galante Negri

Outline

Linear Regression

- Performs predictions through the weighted sum of the feature vector components (plus an additional "bias" term)
- Given $\mathbf{x} = [x_1, \ldots, x_n]^T \in \mathcal{X} \subseteq \mathbb{R}^n$, a Linear Regression model assumes the form:

$$\hat{y}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n, \tag{1}$$

- It is common to rewrite the model in matrix form:

$$\hat{y}(\mathbf{x}) = \theta^T \mathbf{x}', \tag{2}$$

where $\theta = [\theta_0, \theta_1, \ldots, \theta_n]^T$ and $\mathbf{x}' = [1, x_1, \ldots, x_n]^T$.

Linear Regression (cont.1)

- Fitting a regression model involves obtaining the appropriate value for each parameter θ_j, for $j = 0, 1, \ldots, n$
- To this end, information from $\mathcal{D} = \{(\mathbf{x}_i, y_i) \in \mathcal{X} \times \mathbb{R} : i = 1, \ldots, m\}$ is used
- Once the regression model is obtained, it becomes possible to estimate a response $\hat{y}$ for any given vector $\mathbf{x}$
- The optimal parameters θ_j are obtained by minimizing the sum of squared errors between estimated and observed values

$$J(\theta) = \sum_{i=1}^{m} \left(\theta^T \mathbf{x}_i - y_i\right)^2 \tag{3}$$

Linear Regression (cont.2)

- The argument θ that minimizes $J(\theta)$ corresponds to $\left.\frac{\partial J(\theta)}{\partial \theta_j}\right|_{j=0,\dots,n} = 0$:

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{\partial \sum_{i=1}^{m} \left(\theta^T \mathbf{x}_i - y_i\right)^2}{\partial \theta_j} = \sum_{i=1}^{m} \left[\frac{\partial \left(\theta^T \mathbf{x}_i - y_i\right)^2}{\partial \theta_j} \right] =$$

$$= \sum_{i=1}^{m} \left[\frac{\partial \left(\theta^T \mathbf{x}_i - y_i\right)^2}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\theta_j} \right] = \sum_{i=1}^{m} [2\mathbf{z} x_{ij}] = 2 \sum_{i=1}^{m} \left[x_{ij} \cdot \left(\theta^T \mathbf{x}_i - y_i\right) \right] = 0,$$

where $\mathbf{z} = \theta^T \mathbf{x}_i - y_i$ was adopted for convenience

- Reorganizing in matrix form:

$$\mathbf{X}^T \left(\mathbf{X}\theta^T - \mathbf{Y}\right) = 0 \Leftrightarrow \mathbf{X}^T\mathbf{X}\theta^T = \left(\mathbf{X}^T\mathbf{Y}\right) \Leftrightarrow \theta^T = \left(\mathbf{X}^T\mathbf{X}\right)^{-1} \left(\mathbf{X}^T\mathbf{Y}\right), \quad (4)$$

where $\mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1n} \\ 1 & x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix}$, $\mathbf{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$

Gradient Descent (once again)

- Another alternative for minimizing the previous problem involves performing iterative adjustments on θ
- The local gradient of $J(\theta)$ with respect to the parameters is observed

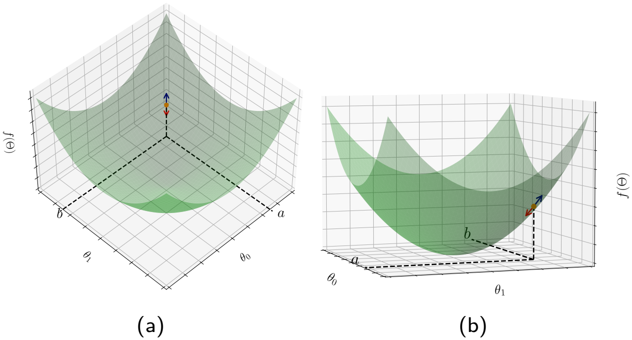


Figure: Gradient of f at coordinate $[a, b]$, represented by the black vector, which points toward the direction of steepest ascent. The direction of steepest descent, opposite to the previous vector, is indicated by the red vector.

Gradient Descent (once again)

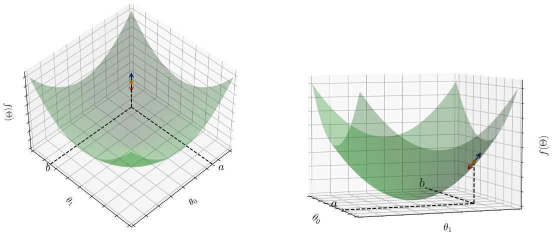
- Based on the previous development, we know that:

$$\nabla J(\theta) = \left[\frac{\partial J(\theta)}{\partial \theta_0}, \ldots, \frac{\partial J(\theta)}{\partial \theta_1} \right] = 2\mathbf{X}^T (\mathbf{X}\theta - \mathbf{Y}) \tag{5}$$

- Therefore, the process reduces to:

$$\theta := \theta - \eta \nabla J(\theta), \tag{6}$$

where $\eta \in \mathbb{R}_+$ is the learning rate



(a) (b)

Outline

Polynomial Regression

- The use of linear models is restricted to data with a linear trend
- However, the formalizations can be reused after data remapping

Construction/example

- Suppose a regression problem based on observations $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)\}$, whose feature vectors are defined in a two-dimensional space and y is a scalar
- A linear model in this space has the form:

$$\hat{y}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

- Suppose further that these data show a cubic trend. Thus, the appropriate regression model can be expressed as:

$$\begin{aligned}\hat{y}(\mathbf{x}) = \theta_0 + \theta_1 x_1^3 + \theta_2 x_2^3 + \theta_3 x_1^2 x_2 + \theta_4 x_1 x_2^2 + \\ + \theta_5 x_1^2 + \theta_6 x_2^2 + \theta_7 x_1 x_2 + \theta_8 x_1 + \theta_9 x_2\end{aligned} \tag{7}$$

Polynomial Regression (cont.1)

Construction/example...

- Considering the following variable changes:

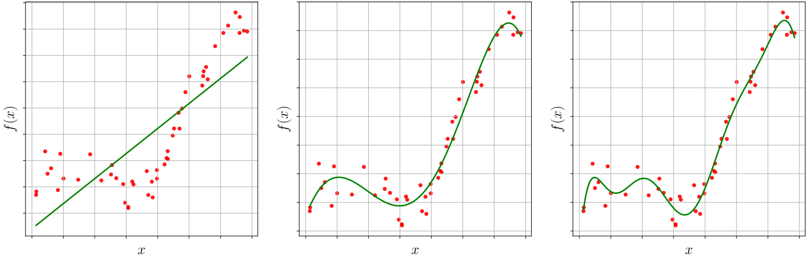
$$\begin{array}{ccc} z_1 = x_1^3 & z_2 = x_2^3 & z_3 = x_1^2 x_2 \\ z_4 = x_1 x_2^2 & z_5 = x_1^2 & z_6 = x_2^2 \\ z_7 = x_1 x_2 & z_8 = x_1 & z_9 = x_2 \end{array}$$

- The cubic model previously presented is rewritten as:

$$\begin{aligned} \hat{y}(\mathbf{x}) = \theta_0 + \theta_1 z_1 + \theta_2 z_2 + \theta_3 z_3 + \theta_4 z_4 + \\ + \theta_5 z_5 + \theta_6 z_6 + \theta_7 z_7 + \theta_8 z_8 + \theta_9 z_9 \end{aligned}$$

- This leads to the conclusion that the simple mapping $[x_1, x_2] \mapsto [x_1^3, x_2^3, \ldots, x_1 x_2, x_1, x_2]$ transforms a cubic model into a linear one

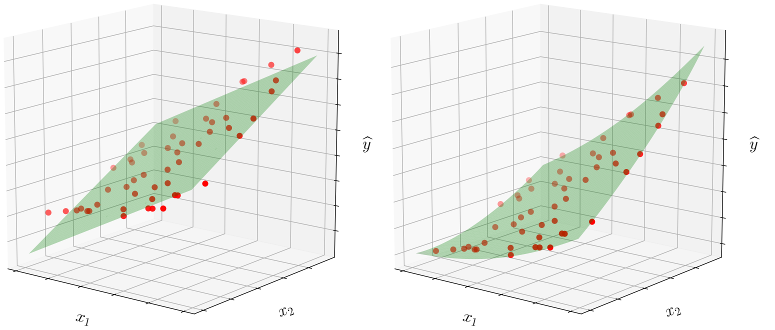
Regression Examples



(a) Linear (b) Polynomial of degree 5 (c) Polynomial of degree 10

Figure: Fits obtained with linear and polynomial regression.

Regression Examples (multiple predictor variables)



(a) Linear (b) Quadratic polynomial

Figure: Fits obtained with linear and quadratic polynomial regression in a problem involving two predictor variables.

Outline

Ridge Regression

- An alternative for controlling overfitting effects in models is through regularization
- The main difference with respect to the previous discussions refers to the inclusion of the term $\alpha \sum_{i=1}^{n} \theta_i^2$ in the objective function, where $\alpha \in \mathbb{R}_+$ is known as the regularization parameter
- Under this consideration, the cost function takes the form:

$$J(\theta) = \sum_{i=1}^{m} \left(\theta^T \mathbf{x}_i - y_i\right)^2 + \alpha \sum_{j=1}^{n} \theta_j^2 \tag{8}$$

- The optimal configuration for θ corresponds to the solution of the equation $\frac{\partial J(\theta)}{\partial \theta} = 0$, which leads to the following expression

$$\theta^T = \left(\mathbf{X}^T\mathbf{X} + \alpha \mathbf{I}_0\right)^{-1} \cdot \left(\mathbf{X}^T\mathbf{Y}\right) \tag{9}$$

Lasso Regression

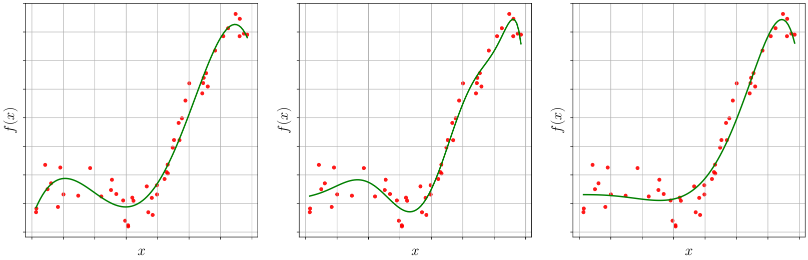
- Lasso – Least Absolute Shrinking and Selection Operator
- The regularization term incorporated into the objective function consists of the sum of the absolute values of the model parameters
- The objective function becomes:

$$J(\theta) = \sum_{i=1}^{m} \left(\theta^T \mathbf{x}_i - y_i\right)^2 + \alpha \sum_{j=1}^{n} |\theta_j| \tag{10}$$

- $J(\cdot)$ is no longer differentiable at $\theta_j = 0$, for $j = 1, \ldots, n$
- Nevertheless, this function can be minimized using Gradient Descent associated with the notion of subgradient at $\theta_j = 0$, as follows:

$$\widetilde{\nabla} J(\theta) = \nabla J(\theta) + \alpha \, \mathrm{sgn}(\theta) \tag{11}$$

Ridge and Lasso Examples



(a) Polynomial of degree 5 (b) Ridge (c) Lasso

Figure: Fits obtained using Ridge and Lasso models, assuming $\alpha = 1.0$. The original data were remapped according to a fifth-degree polynomial.

Outline

Logit Regression

- Widely employed in estimating the probability of "element-class" membership in binary association problems
- Let $\mathcal{D} = \{(\mathbf{x}_i, y_i) \in \mathcal{X} \times \mathbb{R} : i = 1, \ldots, m\}$, with $y_i \in \{0, 1\}$
- This model can be understood as the composition of a linear model with a sigmoid function:

$$g(\mathbf{x}; \theta) = \frac{1}{1 + e^{-\left(\theta^T \mathbf{x}\right)}} \tag{12}$$

- The outputs of this function can be interpreted as the probability of $\mathbf{x}$ belonging to the class labeled as 1
- In this context, the cost associated with a single observation $\mathbf{x}$ can be modeled as:

$$h(\mathbf{x}_i, \theta) = \begin{cases} -\log\left(g(\mathbf{x}_i; \theta)\right), & \text{if } y_i = 1 \\ -\log\left(1 - g(\mathbf{x}_i; \theta)\right), & \text{if } y_i = 0 \end{cases} \tag{13}$$

Logit Regression

- This leads to the following objective function:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y_i \cdot \log\left(g(\mathbf{x}_i;\theta)\right) + (1-y_i)\cdot \log\left(1-g(\mathbf{x}_i;\theta)\right)\right] \tag{14}$$

- The derivative of Equation (19) with respect to θ does not provide an analytical closed-form solution
- However, it has a convex form, allowing then convergence through optimization algorithms such as Gradient Descent (or similar methods)
- In this case, the j-th component of the gradient vector is computed as:

$$\frac{\partial J(\theta)}{\partial \theta_i} = \frac{1}{m}\sum_{i=1}^{m}\left[\left(\frac{1}{1+e^{-\left(\theta^T \mathbf{x}\right)}} - y_i\right) x_{ij}\right]. \tag{15}$$

Examples of Logit Regression

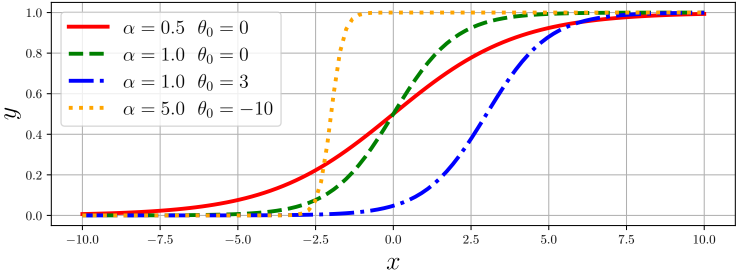


Figure: Examples of sigmoid functions in a one-dimensional case under different parameterizations. Suppose $\alpha = \theta_1$ on this one-dimensional model.

Examples of Logit Regression (cont.1)

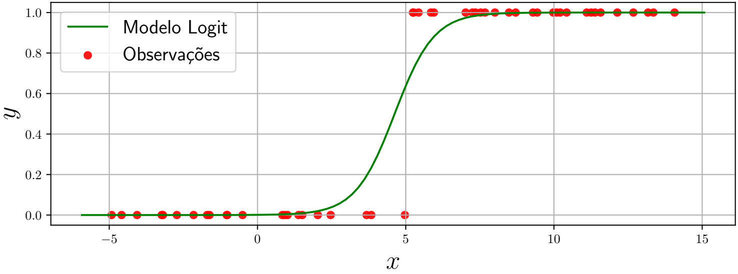


Figure: Example of Logit regression.

Softmax Regression

- Intended for problems involving a larger number of classes
- For a given vector $\mathbf{x}_i$, a "score" is computed with respect to each class, $k = 1, 2, \ldots, c$, involved in the problem through the following function:

$$h(\mathbf{x}_i; k, \Theta) = \theta_k^T \mathbf{x}_i \tag{16}$$

  where θ_k is the parameter vector responsible for fitting the model with respect to the class indicated by k
- The matrix Θ, of order $c \times (n+1)$, organizes the parameter vectors of the different classes, where θ_k occupies the k-th row
- Once $h(\mathbf{x}_i; k, \Theta)$ is computed for $k = 1, 2, \ldots, c$, the class-membership probability is obtained through:

$$p(\mathbf{x}_i; k, \Theta) = \frac{e^{h(\mathbf{x}_i; k, \theta)}}{\sum_{j=1}^{c} e^{h(\mathbf{x}_i; j, \theta)}} \tag{17}$$

Softmax Regression (cont.1)

- Once the class-membership probabilities are obtained, the final choice is determined by maximizing the probability of occurrence, that is:

$$\widehat{y} = \arg\max_{k} p(\mathbf{x}_i; k, \Theta) \tag{18}$$

- The main issue that arises concerns determining the matrix Θ
- The cost function that allows its estimation is expressed in terms of "cross-entropy":

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{c} \delta_j(y_i) \log\left(p(\mathbf{x}_i; j, \Theta)\right), \tag{19}$$

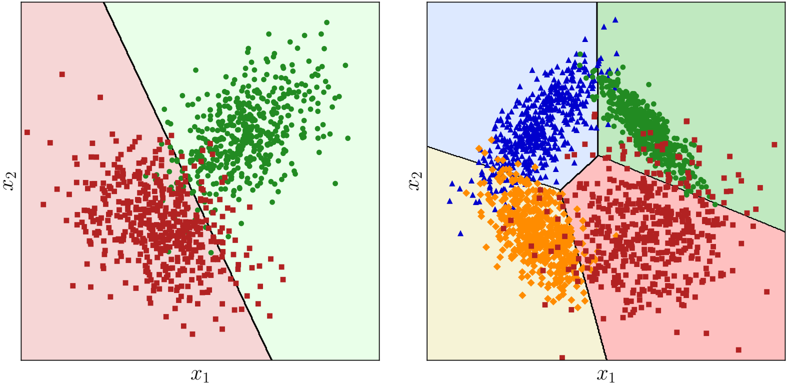
where $\delta_j(y_i) = \begin{cases} 1 \text{ if } y_i = j \\ 0 \text{ if } y_i \neq j \end{cases}$.

- Furthermore:

$$\nabla_k J(\Theta) = \frac{1}{m} \sum_{i=1}^{m} \left(p(\mathbf{x}_i; k, \Theta)\right) - \delta_k(y_i)\mathbf{x}_i, \tag{20}$$

which enables the optimization of J through Gradient Descent or another optimization method

Classification via Logit and Softmax



(a) Logit Regression (b) Softmax Regression

Figure: Examples of binary and multiclass classification using Logit and Softmax models.

Outline

Regression via SVM

- The SVM method can also be adapted to regression problems
- In this case, the objective is no longer to find a separating surface between classes, but rather to determine a function:

$$g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b \tag{21}$$

- The central idea is to fit $g(\mathbf{x})$ such that deviations with respect to the observed values y_i are tolerated within a margin ϵ
- Thus, errors smaller than ϵ are not penalized
- Errors larger than ϵ are controlled through slack variables ξ

Regression via SVM: ϵ-margin

- Given a set of observations:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i) \in \mathcal{X} \times \mathbb{R} \colon i = 1, \ldots, m\}, \tag{22}$$

- The model must satisfy:
  1. $\left(\mathbf{w}^T\mathbf{x}_i + b\right) - y_i \leq \epsilon + \xi_i$,
  2. $y_i - \left(\mathbf{w}^T\mathbf{x}_i + b\right) \leq \epsilon + \hat{\xi}_i$.
- The variables ξ_i and $\hat{\xi}_i$ control positive and negative deviations exceeding the tolerance ϵ.
- The parameter ϵ defines an error-insensitive region

Regression via SVM: primal problem

Primal formulation

$$\min_{\mathbf{w},b} \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{m}\left(\xi_i + \hat{\xi}_i\right)$$
$$\text{subject to: } \begin{cases} \left(\mathbf{w}^T\mathbf{x}_i + b\right) - y_i \leq \epsilon + \xi_i \\ y_i - \left(\mathbf{w}^T\mathbf{x}_i + b\right) \leq \epsilon + \hat{\xi}_i \\ \xi_i, \hat{\xi}_i \geq 0;\ \ i = 1, \dots, m \end{cases} \tag{23}$$

- The term $\frac{1}{2}\mathbf{w}^T\mathbf{w}$ controls model complexity
- The parameter C controls the penalization of deviations larger than ϵ

Regression via SVM: dual form

Dual problem

$$\max_{\lambda_i,\hat{\lambda}_i} L_D(\lambda_i,\hat{\lambda}_i)$$
$$\text{subject to: } \begin{cases} \sum_{i=1}^{m}\left(\lambda_i-\hat{\lambda}_i\right)=0 \\ 0\le \lambda_i,\hat{\lambda}_i\le C;\ \ i=1,\dots,m \end{cases} \tag{24}$$

where

$$L_D(\lambda_i,\hat{\lambda}_i)=-\frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\left(\lambda_i-\hat{\lambda}_i\right)\left(\lambda_j-\hat{\lambda}_j\right)\mathbf{x}_i^T\mathbf{x}_j+\sum_{i=1}^{m}\left(\hat{\lambda}_i-\lambda_i\right)y_i-\epsilon\sum_{i=1}^{m}\left(\hat{\lambda}_i+\lambda_i\right)$$

- The solution allows writing:

$$\mathbf{w}=\sum_{i=1}^{m}(\hat{\lambda}_i-\lambda_i)\mathbf{x}_i \tag{25}$$

- Therefore, the regression model becomes:

$$g(\mathbf{x})=\sum_{i=1}^{m}(\hat{\lambda}_i-\lambda_i)\mathbf{x}_i^T\mathbf{x}+b \tag{26}$$

Regression via SVM and kernels

- As in the classification case, the inner product can be replaced by a kernel function:

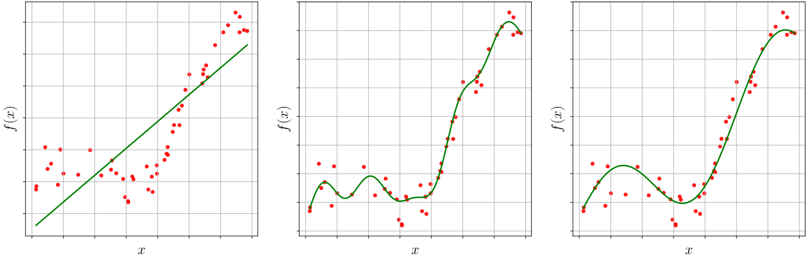
$$g(\mathbf{x}) = \sum_{i=1}^{m} (\hat{\lambda}_i - \lambda_i) K(\mathbf{x}_i, \mathbf{x}) + b \tag{27}$$

- This makes it possible to obtain nonlinear regression models
- Common examples (already known):
  - Linear kernel
  - Polynomial kernel
  - RBF kernel
- For example, the parameter γ of the RBF kernel controls the flexibility of the fit

Regression via SVM: parameter interpretation

- C: controls the penalization of errors larger than ϵ
  - High C: more rigid fit to the data
  - Low C: more regularized model
- ϵ: defines the width of the error-insensitive region
  - High ϵ: greater tolerance to deviations
  - Low ϵ: fit becomes more sensitive to the data
- γ: in the RBF kernel, controls the flexibility of the estimated function
  - High γ: more local fit
  - Low γ: smoother fit

Examples of SVM Regression ($C = 100, \epsilon = 10^{-3}$)



(a) Linear (b) RBF – $\gamma = 0.1$ (c) RBF – $\gamma = 1.0$

Figure: Regression with SVM.

Outline

Regression via CART

- The CART method can also be adapted for regression problems
- In classification, tree construction is guided by impurity reduction
- In regression, node splitting is guided by reducing deviations between observed values and estimated mean values
- The tree is still constructed through successive decisions over the attributes of the patterns
- However, each leaf stores a numerical value instead of a class label

Regression via CART: central idea

- Consider a local subset of observations $\mathcal{Q}$
- The response associated with this subset can be represented by the mean value:

$$y_{\mathcal{Q}} = \frac{1}{\#\mathcal{Q}} \sum_{i=1}^{\#\mathcal{Q}} y_i \qquad (28)$$

- The quality of this approximation is measured by the squared deviations:

$$D(\mathcal{Q}) = \sum_{i=1}^{\#\mathcal{Q}} \left(y_i - y_{\mathcal{Q}}\right)^2 \qquad (29)$$

- The objective is to find splits that reduce this deviation

Regression via CART: deviation reduction

Reduction provided by a threshold τ_{kh}

$$\Delta D(\mathcal{Q};\tau_{kh}) = D(\mathcal{Q}) - \frac{\#\mathcal{Q}_{inf}(\tau_{kh})}{\#\mathcal{Q}} D(\mathcal{Q}_{inf}(\tau_{kh})) - \frac{\#\mathcal{Q}_{sup}(\tau_{kh})}{\#\mathcal{Q}} D(\mathcal{Q}_{sup}(\tau_{kh})) \quad (30)$$

- The best threshold is the one that provides the largest reduction in deviations
- The process is recursively applied until some stopping criterion is satisfied
- Each leaf returns the mean value of the observations that reached it

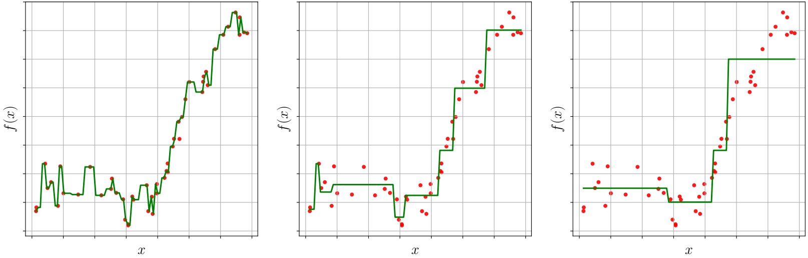
Regression via CART: leaf and prediction

- After training, each tree leaf represents a region of the feature space
- For a new observation $\mathbf{x}$:
  1. $\mathbf{x}$ traverses the sequence of tree decisions
  2. a specific leaf is reached
  3. the prediction is given by the mean value stored in that leaf
- Thus, the CART regression model produces a piecewise function
- The prediction is constant within each region defined by the tree

Regression via CART: parameters

- `min_samples_split`:
  - controls the minimum number of observations required to split a node
  - low values produce more flexible trees
  - high values produce more general models
- `min_impurity_decrease`:
  - controls the minimum reduction required to accept a split
  - acts as a stopping criterion
  - helps avoid overfitting
- Excessive reduction of these parameters may lead to poorly generalizable models

Examples of CART Regression ($\zeta = 10^{-7}$)



(a) $\psi = 2$ (b) $\psi = 10$ (c) $\psi = 20$

Figure: Regression with CART.

SVR vs. CART for Regression

Aspect	**SVR**	**CART**
Model	Global, possibly kernelized function	Piecewise function
Nonlinearity	Obtained through kernels	Obtained through successive splits
Regularization	Controlled by C, ϵ, and the kernel	Controlled by depth and stopping criteria
Sensitivity to scale	High	Low
Interpretability	Less direct	More intuitive
Risk of overfitting	High with large C or γ	High with very deep trees

Random Forest for Regression

- The Random Forest, as we know, is an *ensemble* of CART regression trees
- Each tree is independently trained using:
  - random subsets of observations (*bootstrap*)
  - random subsets of attributes
- Let $\mathcal{P}_k(\mathbf{x})$ denote the prediction produced by the k-th tree
- In regression, the final model output is obtained through the average of the individual predictions:

$$\hat{y}(\mathbf{x}) = \frac{1}{K} \sum_{k=1}^{K} \mathcal{P}_k(\mathbf{x}), \tag{31}$$

  where K represents the number of trees in the forest
- Each CART tree is still constructed through the minimization of local squared deviations:

$$D(\mathcal{Q}) = \sum_{i=1}^{\#\mathcal{Q}} (y_i - y_{\mathcal{Q}})^2 \tag{32}$$

- The combination of multiple trees reduces variance and tends to improve the model generalization capability

Regression Models

rogerio.negri@unesp.br